

GOVERNMENT SUBSIDY & GRANT DISBURSEMENT TRACKING SYSTEM

Milestone 2 — Team Guide

Eligibility Scoring & Verification Workflow Development

Weeks 3 & 4 · Read this before you start coding

WHAT YOU ARE BUILDING IN THIS MILESTONE

By the end of Week 4, a complete application journey must work: a citizen submits an application, the system automatically scores their eligibility, and the application moves through three approval levels — Field Officer, District Officer, Finance Approver — before the grant is approved and a disbursement plan is triggered.

Weeks	Week 3 and Week 4
Objective	Build beneficiary/scheme modules, eligibility scoring engine, and multi-level verification workflow
End goal	A working end-to-end application submission and approval chain
Key milestone	Finance Approver's final approval must automatically create a disbursement plan (links to Milestone 3)

TASK 1 — Beneficiary & Scheme Master Data

What you need to build

- Beneficiary registration — save full profile: name, Aadhaar, income level, category, district, contact details
- Document linking — attach uploaded documents to the correct beneficiary record
- Duplicate check — if the same Aadhaar number already exists in the system, return an error (HTTP 409)
- Scheme management — create and list schemes including their eligibility criteria
- Eligibility criteria must be stored in the database — never hardcoded in Java

Database tables involved

beneficiary	Stores every applicant's personal details and category
document	Stores uploaded files, linked by beneficiary_id

scheme	Stores scheme name, budget, grant limits, active dates
eligibility_rules	Stores scoring criteria per scheme — income cap, category match, etc.

IMPORTANT — DO NOT HARDCODE ELIGIBILITY RULES

Wrong: `if (category.equals("FARMER")) { score += 40; }` Correct: read the criteria from the `eligibility_rules` table in the database. This way, when a scheme's rules change, only the database row changes — not your code.

Self-check before moving to Task 2

- Register a test beneficiary — does it save to the database correctly?
- Try registering the same Aadhaar number twice — does it return HTTP 409?
- Create a new scheme with income and category criteria — are they stored in `eligibility_rules`, not in code?
- Can you retrieve a beneficiary's full profile including linked documents via `GET /api/v1/beneficiaries/{id}`?

TASK 2 — Eligibility Scoring Engine

How the scoring works

When a citizen submits an application, the system automatically calculates an eligibility score out of 100. This score is based on three criteria:

Criterion	Max Points	What it checks	Example
Income Level	30 pts	Household income vs scheme's maximum income cap	Income < ₹1L = 30 pts. Income ₹1–3L = 20 pts.
Category Match	40 pts	Does applicant's category match the scheme?	FARMER applying for PM Kisan = 40 pts. STUDENT = 0 pts.
Document Completeness	30 pts	How many required documents are uploaded?	4 of 4 docs = 30 pts. 2 of 4 docs = 15 pts.

Score ≥ 60 (configurable) = ELIGIBLE — application moves to Field Officer. Score < 60 = AUTO-REJECTED — applicant is notified with reason.

What you need to build

- POST /api/v1/applications — application submission endpoint
- Score calculation — reads criteria from eligibility_rules table, calculates total score
- Score log — save each criterion's weight and score obtained to eligibility_score_log table (one row per criterion, not just a total)
- Routing logic — score $\geq$ threshold: create verification_workflow record assigned to Field Officer. Score $<$ threshold: mark application REJECTED
- API response must include the full score breakdown, not just the total number

Database tables involved

application	One row per submission — stores score, status, current stage
eligibility_score_log	One row per criterion per application — the detailed breakdown
verification_workflow	Created when application passes threshold — assigned to Field Officer

TECHNICAL REQUIREMENT — @Transactional

The entire submission — saving the application, writing the score log, and creating the workflow assignment — must be wrapped in @Transactional. If any part fails, everything rolls back. A half-saved application is worse than no application at all.

Self-check before moving to Task 3

- Submit an application where income is too high — does it auto-reject?
- Submit one where all criteria pass — does a verification_workflow row appear in the database?
- Check the eligibility_score_log table — is there one row per criterion (income, category, documents), not just a total?
- Does the API response show the breakdown of how the score was reached?
- If you change the scoring threshold in the database from 60 to 70 and resubmit, does the result change — without changing any code?

TASK 3 — Multi-Level Verification Workflow

The three-level approval chain

Once an application passes the eligibility threshold, it enters a human verification chain. Each level can take one of four actions:

Level	Role	Available actions
Level 1	Field Officer	APPROVE (moves to District Officer) / REJECT / ESCALATE (skip to District Officer) / RE_VERIFY (send back to applicant)
Level 2	District Officer	APPROVE (moves to Finance Approver) / REJECT / ESCALATE (skip to Finance Approver) / RE_VERIFY (send back to Field Officer or applicant)
Level 3	Finance Approver	APPROVE (final — sets grant amount, triggers disbursement plan) / REJECT (closes the case)

What you need to build

- POST /api/v1/workflow/{applicationId}/action — action endpoint accepting: action type, comments, and (for Finance) approved amount
- State transitions — application status and current stage must update correctly after every action
- Role enforcement — @PreAuthorize on every workflow endpoint. A Field Officer cannot act on a District Officer stage application. Must return HTTP 403.
- Escalation — moves application to next level without requiring approval at current level
- Re-verification — sends application back to a previous stage or to the applicant with a mandatory reason
- Notification entry — every action creates a notification so the applicant can see what happened and why
- Audit log entry — every action recorded in audit_log with: user, action, old status, new status, timestamp

Database tables involved

verification_workflow	One row per stage action — who acted, what they did, comments, timestamp
application	Status and current_stage updated after every action
notifications	One entry per action so applicant can track their application
audit_log	Permanent record of every action — cannot be edited or deleted

CRITICAL — SECURITY MUST BE AT API LEVEL

Hiding a button on the screen is NOT security. A Field Officer must not be able to call the Finance Approver's endpoint even directly via Postman with a manually typed URL. Use `@PreAuthorize("hasRole('FINANCE_APPROVER')")` on the service method. Test this by calling the endpoint with the wrong role's JWT token — it must return 403 Forbidden.

Self-check before calling Milestone 2 done

- Submit a test application, get it scored, and walk it through all three approval levels in sequence
- Try using a Field Officer login to act on a District Officer stage application — must return 403
- Reject an application at District Officer level — is the reason recorded? Is the notification created?
- Escalate at Field Officer — does it skip to District Officer correctly?
- Finance Approver final approval — check that a `disbursement_plan` row is created in the database
- Check `audit_log` — is there a row for every single action taken in the workflow?

TASK 4 — Testing & Validation**Why testing matters here specifically**

This milestone handles the rules that determine who gets money and who doesn't. A bug in the scoring engine or a gap in the security layer is not just a technical problem — it is a correctness and fairness problem. Every code path must be tested before Milestone 2 is considered complete.

Test scenarios every team member should run

Test scenario	What to send	Expected result
Full eligibility — all criteria pass	Income < ₹1L, correct category, all 4 docs uploaded	Score = 100, assigned to Field Officer
Partial eligibility — income too high	Income > ₹5L, correct category, all docs uploaded	Score = 70 (category + docs only), auto-reject if < 60
No documents uploaded	Income OK, category match, 0 documents uploaded	Score = 70 (income + category only), verify threshold logic
Wrong category for scheme	STUDENT applying for farmer-only scheme	Score = 60 (income + docs only), test threshold boundary
Duplicate Aadhaar	Register same Aadhaar twice	HTTP 409 Conflict on second attempt

Test scenario	What to send	Expected result
Field Officer approves	Valid Field Officer JWT, APPROVE action	Application moves to District Officer queue
Field Officer acts on wrong stage	Field Officer JWT on District Officer stage application	HTTP 403 Forbidden
District Officer rejects	Valid District Officer JWT, REJECT with reason	Application status = REJECTED, notification created
Escalation at Field Officer	ESCALATE action at Field Officer stage	Application skips to District Officer with ESCALATED flag
Finance Approver final approval	APPROVE with grant amount set	disbursement_plan row created in database
Wrong role on protected endpoint (Postman test)	Field Officer JWT token on Finance Approver endpoint	HTTP 403 Forbidden — not just a missing button

HOW TO TEST — USE POSTMAN

Every scenario above should be tested through Postman, not just through the UI. Call the API endpoint directly with the correct JWT token. Check the database after every call to confirm the data is exactly what you expect — not just what the API response says.

MILESTONE 2 DELIVERABLES

These are the three deliverables that must be complete and working by the end of Week 4:

Deliverable	What complete means	Who is responsible
Beneficiary Management Module	Create, view, and update beneficiary records. Document upload linked correctly. Duplicate Aadhaar rejected. Scheme master data with configurable eligibility criteria.	Shafi (DB structure) + Bhagyashri/Priyanku (backend logic)
Eligibility Scoring & Verification Workflow Engine	Score calculated automatically on submission. Breakdown logged per criterion. Auto-reject below threshold. Score and criteria configurable from database without code change.	Priyanku (primary) + Bhagyashri (supporting)
Functional Multi-Level Verification Process	All three approval levels working. All four actions (Approve, Reject, Escalate, Re-verify) functioning at each level. Role enforcement	Priyanku (API) + Joyal (testing) + Prasanna/Dhanus (officer UI screens)

Deliverable	What complete means	Who is responsible
	at API level. Audit log populated. Finance approval triggers disbursement plan.	

CONNECTING TO MILESTONE 3

When the Finance Approver gives final approval, the system must automatically create a record in the `disbursement_plan` table. This is the handoff point to Milestone 3. If this connection is not in place by end of Week 4, Milestone 3 cannot begin on time. Make this the last thing you test before declaring Milestone 2 complete.